Wormhole Aggregation: Recursive Zero-Knowledge Settlement for Post-Quantum Blockchains

Christopher Smith[1] and Ethan Cemer[1]

[1]Quantus Labs

March 2026

Abstract

Post-quantum signature schemes impose large public keys and signatures, increasing on-chain costs while still leaking transaction structure. We present *Quantus Wormhole*, a private settlement protocol where users deposit to addresses derived from secrets and later prove in zero knowledge that the deposit exists, hasn't been spent, and satisfies fee rules. Recursive aggregation amortizes verification across many withdrawals, while dummy padding and local shuffling enlarge the anonymity set for balance-linking adversaries. We formalize the withdrawal relation, analyze privacy guarantees, prove security properties, and report benchmarks for two-layer aggregation of up to 256 leaf proofs.

1 Introduction

The post-quantum transition poses a dilemma for blockchains. Shor's algorithm breaks elliptic-curve signatures [14], but post-quantum replacements like ML-DSA impose keys and signatures an order of magnitude larger [8]. A naïve swap increases bandwidth and state growth while doing nothing for privacy. Every transaction still reveals sender, recipient, and amount.

One response is to verify post-quantum signatures inside a zero-knowledge circuit and aggregate many such proofs, compressing the on-chain footprint. But this requires implementing post-quantum signature verification in-circuit, which is expensive.

This paper takes a different approach: rather than proving signature validity, users prove knowledge of a secret tied to an unspent deposit. This idea appears in Ziggy [4], a STARK-based signature scheme where signers prove knowledge of a hash preimage. We apply the same principle to blockchain settlement, building on EIP-7503's "wormhole" concept [7]: funds are sent to addresses derived from secrets and later withdrawn by proving knowledge of that secret. This sidesteps post-quantum signature verification entirely. The circuit only needs hash functions and storage-proof verification. We add recursive proof aggregation for scalable settlement [12].

Contributions:

1. A formal specification of the wormhole-address withdrawal relation, covering nullifier derivation, authenticated state inclusion, and fee constraints.

2. A two-layer recursive aggregation architecture where the first layer performs privacy-sensitive dummy padding and shuffling on the client, while subsequent layers can be delegated.

3. Combinatorial analysis of anonymity set size under the first-layer grouping rules.
4. Security proofs for deposit binding, one-time withdrawal, and value conservation, with explicit separation of proof-level and chain-level assumptions.
5. Benchmarks demonstrating 20 ms leaf proving, sub-4 ms verification, and two-layer aggregation scaling to 256 leaves.

2 Background and Design Goals

2.1 Wormholes and Privacy Pools

EIP-7503 proposes *zero-knowledge wormholes*: deposit to secret-derived addresses, then prove knowledge of the secret to withdraw [7]. We extend this concept with post-quantum proving and recursive aggregation with dummy padding for privacy.

Related systems include privacy pools and shielded ledgers like Zcash and Tornado Cash [3, 6, 7, 10]. Unlike fully shielded ledgers, Quantus proves against native chain state without a separate note commitment tree. Importantly, no ciphertexts are stored on-chain, so a future break in cryptographic assumptions cannot retroactively compromise transaction privacy.

2.2 Transparent Recursion

We use Plonky2, combining PLONK arithmetization with FRI commitments for recursive verification without trusted setup [1, 9]. FRI is based on hash functions rather than elliptic curves, making it plausibly post-quantum secure—no known quantum algorithm offers more than a quadratic speedup against hash preimage or collision resistance. Poseidon2 provides efficient in-circuit hashing over prime fields [5].

2.3 Design goals

The protocol aims to satisfy four simultaneous goals:

1. **Amortized Verification:** The marginal on-chain cost of verifying an additional withdrawal should be small.
2. **Deposit Binding:** Every withdrawal must be bound to a real deposit recorded in authenticated chain state.
3. **Replay Prevention:** A deposit should not authorize multiple withdrawals; nullifiers must uniquely bind proofs to deposits.
4. **Meaningful Privacy:** The chain should reveal only the public information needed for settlement: exits, amounts, shared fee parameters, nullifiers, and a block hash. Funding accounts, secrets, and raw storage proofs should remain private.

3 System, Adversary, and Trust Model

3.1 Roles

- **User / Prover:** Generates a secret, derives a wormhole address, deposits funds, and later produces a zero-knowledge proof to withdraw. First-layer aggregation should be performed locally to preserve privacy.

- **Aggregator:** Combines multiple proofs into a single aggregate proof. This role may be the user, a third-party service, or a block producer. Higher-layer aggregation can be delegated since it operates only on public outputs.
- **Blockchain / Verifier:** Validates the final proof, checks nullifier freshness, and settles withdrawals to the specified exit accounts.

3.2 Adversarial capabilities

We assume a probabilistic polynomial-time adversary that can:

- observe the full chain and all public proof outputs,
- submit arbitrary deposits,
- perform timing analysis, and
- attempt to replay or malform proof submissions.

The adversary cannot:

- break the collision resistance of the hash function,
- break the knowledge soundness of the proof system, or
- violate the chain's block-history availability.

3.3 Trust assumptions

The protocol relies on the following assumptions.

1. **Hash Security:** The Poseidon2 instantiation used by the system remains collision resistant and preimage resistant for the relevant security level [5].
2. **Proof Soundness:** The configured Plonky2 recursion stack remains knowledge sound under its standard assumptions [1, 9].
3. **Authenticated History:** The blockchain can check that the public block hash and block number correspond to a valid block in chain history.
4. **Witness Availability:** Honest users can obtain the storage proof needed to build a witness for their deposit.

4 Protocol Overview

The protocol consists of five stages.

1. Secret Generation and Deposit: The user samples a secret s and derives a wormhole address

$$\mathsf{WA}(s) = H(H(\mathsf{salt}_{\mathsf{wh}} \parallel s)).$$

Because this address is derived from a hash rather than a public key, no one can spend from it via ordinary signature-based authorization—funds sent here are effectively burned. Anyone can deposit to a wormhole address through an ordinary on-chain transfer, creating a record that can later be authenticated inside the circuit. A user may receive funds from their own prior transfers or from third-party payments to wormhole addresses derived from secrets the user controls.

2. Witness Acquisition: To withdraw, the user gathers the deposit record and a storage proof. These form the private witness.

3. Leaf Proof Generation: The user chooses up to two exit outputs, computes a nullifier

$$\mathsf{Null}(s, c) = H(H(\mathsf{salt}_{\mathsf{null}} \parallel s \parallel c)),$$

and produces a leaf proof. The proof reveals the nullifier, the exits, the output amounts, the asset identifier, the fee basis points, the block hash, and the block number, while keeping the secret and storage proof private. Leaf proofs are not zero-knowledge. They are only seen by the user who generated them and then included in the first aggregation layer as private inputs.

4. First-Layer Aggregation: The user batches leaf proofs that share the same `block_hash`, pads with dummy leaves if needed, shuffles the order, and produces a zero-knowledge wrapper proof that forwards grouped exits and nullifiers. A user may combine multiple deposits from distinct wormhole addresses (each derived from a different secret) into a single aggregated withdrawal. This step should be performed locally to preserve privacy.

5. Settlement: The blockchain verifies the aggregate proof, checks that the public `block_hash` matches the referenced block number, verifies that the fee parameter matches the configured rate, rejects reused nullifiers, and settles the exits.

5 Formal Specification and Interfaces

5.1 Leaf public inputs

The leaf proof exposes the following public fields:

- `asset_id`: Token identifier.
- `output_amount_1`, `output_amount_2`: Exit amounts (secondary is zero if unused).
- `fee_bps`: Fee parameter in basis points.
- `nullifier`: Replay-prevention tag derived from secret and counter.
- `exit_account_1`, `exit_account_2`: Destination accounts.
- `block_hash`: Commitment to block header fields.
- `block_number`: Height of the referenced block.

5.2 Leaf private witness

The private witness includes:

- `secret`: User secret from which nullifier and wormhole address are derived.
- `counter`: Per-deposit counter used in nullifier derivation.
- `wormhole_account`: Derived wormhole address $\mathsf{WA}(s)$.
- `storage_proof`: Inclusion proof for the deposit record.
- `input_amount`: Deposited amount before fee deduction.

5.3 Leaf relation

Let $x = (a, v_1, v_2, f, n, \alpha_1, \alpha_2, h_B, b)$ denote the public input tuple and let w denote the witness described above. The leaf relation $\mathcal{R}_{\text{leaf}}$ holds when there exists a witness w such that:

(C1) **Nullifier Correctness:** $n = \mathsf{Null}(s, c)$.

(C2) **Wormhole-Address Correctness:** $\alpha_{\text{wh}} = \mathsf{WA}(s)$.

(C3) **Authenticated Inclusion:** The storage proof verifies that the deposit record exists in chain state.

(C4) **Block Binding:** The storage proof is bound to the public block hash h_B and block number b.

(C5) **Fee Constraint:** The disclosed exits satisfy

$$(v_1 + v_2) \cdot 10000 \leq v_{\text{in}} \cdot (10000 - f).$$

The inequality in (C5) allows users to withdraw less than their full balance. This provides a privacy benefit: by forfeiting some value, a user can choose output amounts that blend with more deposits, enlarging their anonymity set. The difference between the deposit and the sum of outputs (beyond fees) is effectively burned.

5.4 On-chain verification

The proof system does not, by itself, settle a withdrawal. The blockchain must also:

- verify the referenced block exists in chain history
- check the fee parameter matches the configured rate
- reject previously-seen nullifiers
- credit exit accounts without increasing total supply

6 Recursive Aggregation Architecture

6.1 First-layer aggregation

The first aggregation layer is privacy-sensitive and should be executed locally. A batch of up to N leaf proofs is aggregated as follows:

- All real leaves must share the same `block_hash`, `asset_id`, and `fee_bps`.
- If fewer than N real leaves are available, the batch is padded with dummy leaves.
- The leaf order is shuffled before wrapping.
- Exits to identical accounts are grouped into a single summed output.
- Real nullifiers are forwarded; dummy nullifiers are replaced by hashing prover-supplied random preimages, preventing a malicious prover from choosing a dummy nullifier that collides with a real one.

A delegated first-layer service would observe raw child proofs before padding and shuffling, collapsing much of the anonymity set quantified in section 7.5.

6.2 Higher-layer aggregation

Higher layers aggregate already-wrapped proofs. Since they operate only on public outputs, they can be safely delegated and do not require zero-knowledge, enabling faster proving.

7 Security and Privacy Analysis

7.1 Deposit binding

Every accepted leaf proof binds to an on-chain deposit: by knowledge soundness of the proof system, there exists a witness satisfying conditions (C1)–(C5), which include storage-proof verification and block binding. The blockchain validates that the public `block_hash`/`block_number` pair refers to a real block, completing the chain of authentication from proof to on-chain state.

7.2 One-time withdrawal

A deposit to a wormhole address creates a transfer record on-chain. To withdraw, a prover must know a secret s such that $\mathsf{WA}(s)$ equals the deposit address; by collision resistance, this secret is unique.

Proposition 1. *If two accepted withdrawals have different nullifiers, they correspond to different deposits.*

Proof. Suppose two withdrawals with nullifiers $n = \mathsf{Null}(s, c)$ and $n' = \mathsf{Null}(s', c')$ correspond to the same deposit. Then both share the same wormhole address and counter, so $\mathsf{WA}(s) = \mathsf{WA}(s')$ and $c = c'$. By collision resistance, $s = s'$. But then $n = n'$, contradicting the assumption. □

Since the blockchain rejects duplicate nullifiers, each deposit can be withdrawn at most once.

7.3 Spend-path exclusivity

A standard address is $\mathsf{Addr}(pk) = H(pk)$ for a public key pk. A wormhole address is $\mathsf{WA}(s) = H(H(\mathsf{salt_{wh}} \parallel s))$.

Proposition 2. *An adversary can withdraw funds from a wormhole address without a valid proof only with negligible probability.*

Proof. To spend from $\mathsf{WA}(s)$ via signature, an adversary must find a keypair (sk, pk) such that $H(pk) = \mathsf{WA}(s)$. This requires either:

1. finding s' such that $H(\mathsf{salt_{wh}} \parallel s')$ equals a public key the adversary controls, or
2. finding pk such that $H(pk) = \mathsf{WA}(s)$.

Both reduce to preimage attacks: (1) requires finding a preimage of a target public key, and (2) requires finding a preimage of a target wormhole address. By preimage resistance, both succeed with negligible probability. Note that for post-quantum signature schemes where $|pk| > |H|$ (e.g., ML-DSA), case (1) is impossible since a hash output cannot equal a larger public key. In this case, a user can prove an address is a wormhole by revealing the inner hash $H(\mathsf{salt_{wh}} \parallel s)$: its length proves it is not a valid public key, without revealing s. □

7.4 Value conservation

Value conservation follows from three properties:

- Each leaf proof enforces the fee constraint (C5): outputs cannot exceed the deposit amount minus fees.
- The wrapper's exit grouping preserves total amounts. It only combines outputs to the same destination.
- One-time withdrawal (above) prevents double-counting the same deposit.

Together with correct on-chain settlement, these ensure that total settled value never exceeds total deposited value minus fees.

7.5 Anonymity sets

An adversary observing a first-layer aggregate sees N nullifiers (some real, some dummies) and grouped exits. Real and dummy nullifiers are both double hashes of high-entropy inputs and are computationally indistinguishable. The adversary cannot determine which nullifiers correspond to real deposits.

Definition 1 (Anonymity set)**.** *Fix a first-layer aggregate Y with total exit amount S and fee rate f. Let $a_1, \ldots, a_M$ be the amounts of all deposits to addresses that have never had an outgoing transaction. The* anonymity set $\mathcal{A}(Y)$ *is*

$$\mathcal{A}(Y) = \left\{ I \subseteq \{1, \ldots, M\} \;\middle|\; |I| \leq N \text{ and } \sum_{i \in I} a_i \geq \frac{S}{1-f} \right\}.$$

Computing $|\mathcal{A}(Y)|$ is an instance of #SubsetSum: given M integers, count subsets whose sum meets a target.

Because the proof system enforces an inequality rather than exact equality (condition C5), the adversary must consider any subset of deposits whose sum is *at least* the required threshold $T = S/(1-f)$. This is a feature: by withdrawing less than their full balance, users can select output amounts that match more potential deposit combinations, enlarging the anonymity set at the cost of burned value.

Complexity of #SubsetSum: The decision version of subset sum (does any subset sum to exactly T?) is NP-complete. The counting version #SubsetSum is #P-complete [13], and no polynomial-time algorithm is known.

Our setting requires counting subsets of size at most N summing to at least T. The size of $|\mathcal{A}(Y)|$ depends on the distribution of deposit amounts relative to T; when many deposits have amounts in the range of typical withdrawals, the anonymity set grows combinatorially. As long as $|\mathcal{A}(Y)| > 1$, an adversary cannot determine which subset is correct without knowing the secret.

Burn Cost and Bounded Rationality: A rational adversary may assume users are unwilling to burn significant value for privacy. Under bounded rationality, the effective anonymity set shrinks to subsets summing to at most $T \cdot (1 + \delta)$ for some small δ (e.g., 1%). Even with this constraint, the anonymity set remains combinatorially large when deposit amounts cluster near typical withdrawal sizes. The protocol does not enforce any upper bound; users may choose their privacy-value tradeoff freely.

N leaf proofs	Prove (s)	Verify (ms)
2	1.55	2.75
4	2.81	2.88
8	5.39	3.02
16	10.74	3.16
32	21.71	3.55

Table 1: First-layer aggregation times for N leaf proofs.

M first-layer proofs	Total leaves	Prove (s)	Verify (ms)
2	16	0.92	2.72
4	32	1.88	2.86
8	64	3.84	3.03
16	128	7.97	3.24
32	256	16.69	3.74

Table 2: Second-layer aggregation times. Each first-layer proof aggregates 8 leaf proofs.

7.6 Privacy leakage

Output amounts, exit addresses, nullifiers, and timing are public. First-layer padding enlarges the anonymity set, but this is plausible deniability, not full shielding. Amount patterns, address reuse, and timing correlation remain potential deanonymization vectors.

8 Evaluation

8.1 Methodology

Benchmarks were executed on an Apple M2 Max with 12 CPU cores and 32 GB of RAM. Circuit artifacts were pre-generated; reported times measure only proving and verification. Prove times include witness assignment and proof generation. Verify times measure verification of a pre-generated proof.

All first-layer measurements use repeated identical dummy leaf proofs with the same `block_hash`. Second-layer benchmarks use repeated first-layer proofs.

8.2 Leaf proofs

Individual leaf proof generation takes approximately 20 ms, with verification in 1.6 ms. These times are dominated by proving; witness assignment is negligible.

8.3 First-layer aggregation

Proving time scales with batch size, while verification remains in the millisecond range across all tested configurations.

8.4 Second-layer aggregation

Second-layer verification time is effectively constant regardless of the number of aggregated proofs, demonstrating the benefit of recursive composition.

9 Related Work

Wormholes and Privacy Pools: EIP-7503 proposes proof-of-burn reminting via secret-derived addresses [7]. Privacy Pools adds compliance-aware privacy sets [3]. Quantus implements recursive proofs over authenticated chain state.

Shielded Systems: Zcash provides strong privacy through shielded notes; Tornado Cash popularized mixer-style privacy on Ethereum [6, 10]. Quantus offers lighter-weight private settlement anchored in native chain state.

Recursive Proofs: Halo and Plonky2 are standard recursive proof systems [2, 9]. Plonky3 provides Poseidon2 primitives with a separate recursion track [11]. We use Plonky2 with Poseidon2 for recursive composition.

Hash-Based Authorization: The Ziggy signature scheme [4] uses a ZK-STARK to prove knowledge of a hash preimage, replacing lattice-based signatures with hash-based proofs.

Post-Quantum Blockchains: Adoption of post-quantum signatures like ML-DSA and SLH-DSA raises questions about transaction size and verification cost [8]. Quantus compresses authorization into ZK proofs rather than publishing large signatures on-chain.

10 Conclusion

Wormhole addresses let a post-quantum blockchain move authorization off-chain while preserving authenticated settlement. Users deposit to secret-derived addresses, prove the deposit exists and hasn't been spent, and reveal only settlement outputs. Recursive aggregation amortizes verification. We formalized the leaf relation, analyzed first-layer privacy, proved security properties, and reported benchmark timings.

References

[1] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In *45th International Colloquium on Automata, Languages, and Programming (ICALP)*, 2018. https://eprint.iacr.org/2017/568

[2] Sean Bowe, Jack Grigg, and Daira Hopwood. Recursive Proof Composition without a Trusted Setup. Cryptology ePrint Archive, Report 2019/1021, 2019. https://eprint.iacr.org/2019/1021

[3] Vitalik Buterin, Jacob Illum, Matthias Nadler, Fabian Schär, and Ameen Soleimani. Blockchain Privacy and Regulatory Compliance: Towards a Practical Equilibrium. SSRN, 2023. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4563364

[4] Lior Gold and Michael Riabzev. Ziggy: A Zero-Knowledge Signature Scheme Based on STARKs. StarkWare, 2020. https://github.com/starkware-libs/ethSTARK

[5] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A Faster Version of the Poseidon Hash Function. Cryptology ePrint Archive, Report 2023/323, 2023. https://eprint.iacr.org/2023/323

[6] Daira Hopwood, Sean Bowe, Taylor Hornby, and Nathan Wilcox. Zcash Protocol Specification. Protocol specification, 2023. https://zips.z.cash/protocol/protocol.pdf

[7] Keyvan Kambakhsh, Hamid Bateni, Amir Kahoori, and Nobitex Labs. EIP-7503: Zero-Knowledge Wormholes. Ethereum Improvement Proposal, 2023. https://eips.ethereum.org/EIPS/eip-7503

[8] National Institute of Standards and Technology. Module-Lattice-Based Digital Signature Standard (FIPS 204). Federal Information Processing Standards Publication 204, 2024. https://csrc.nist.gov/pubs/fips/204/final

[9] Polygon Zero Team. Plonky2: Fast Recursive Arguments with PLONK and FRI. Software repository, 2022. https://github.com/0xPolygonZero/plonky2

[10] Alexey Pertsev, Roman Semenov, and Roman Storm. Tornado Cash Privacy Solution. Project documentation, 2019. https://tornado.cash

[11] Plonky3 Contributors. Plonky3. Software repository, 2026. https://github.com/Plonky3/Plonky3

[12] Quantus Network. qp-zk-circuits. Software repository, 2026. https://github.com/quantus-network/qp-zk-circuits

[13] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[14] Peter W. Shor. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.